



COM3021 – Data Science at Scale

Distributed Machine Learning

DR YANDA MENG



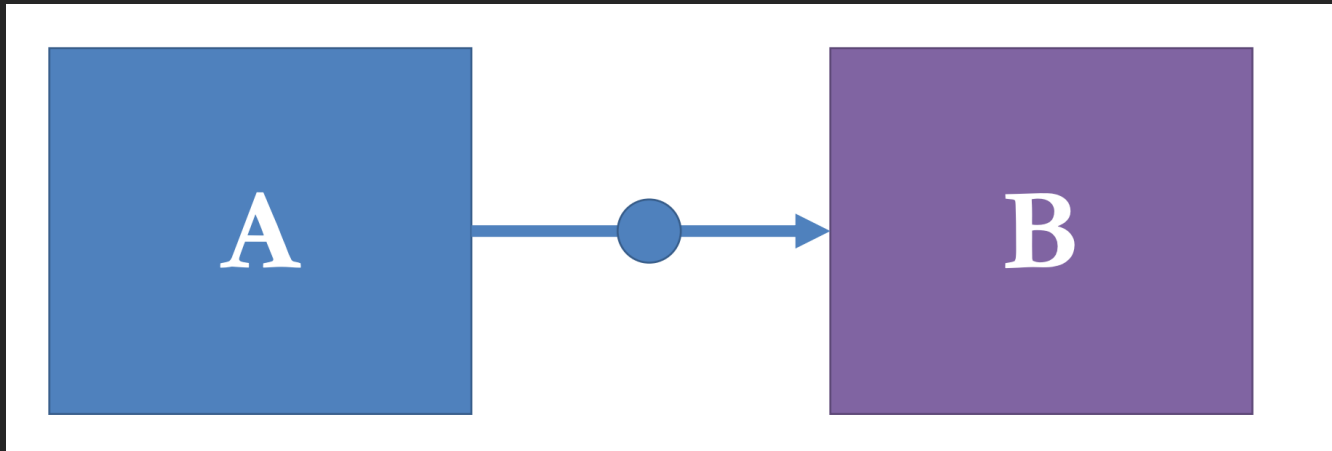
Introduction

- One of the most important aspects to consider when implementing distributed algorithms is the patterns of communication.
- One thing we want to avoid is for one machine to be waiting idle for another machine.

Communication patterns

- Push

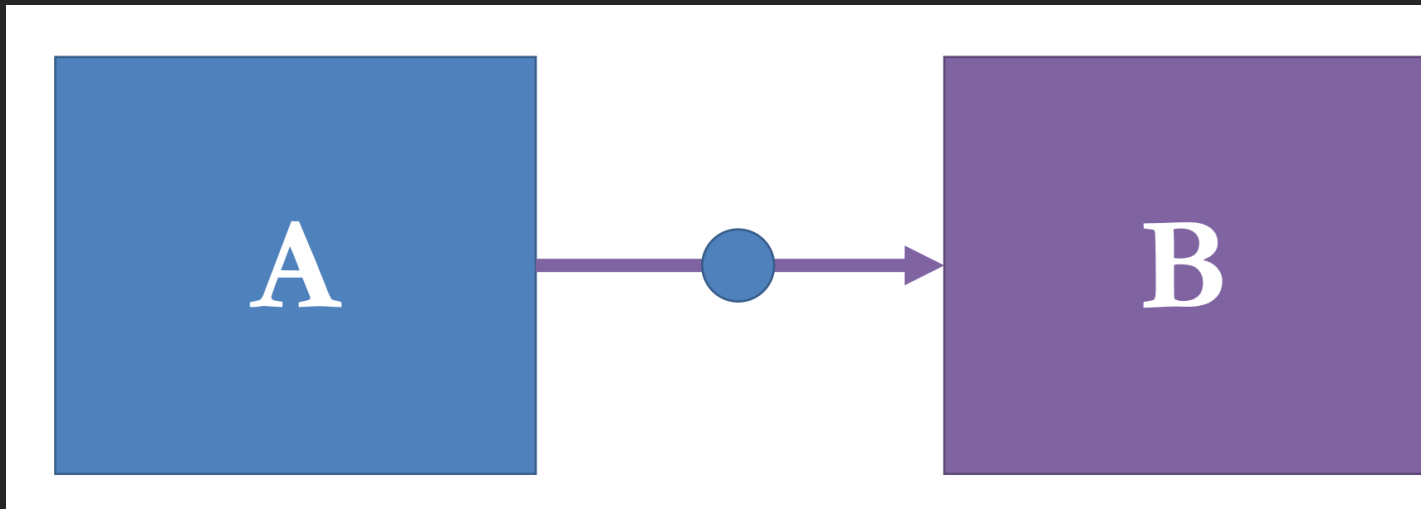
Machine A sends data to Machine B



Communication patterns

- Pull

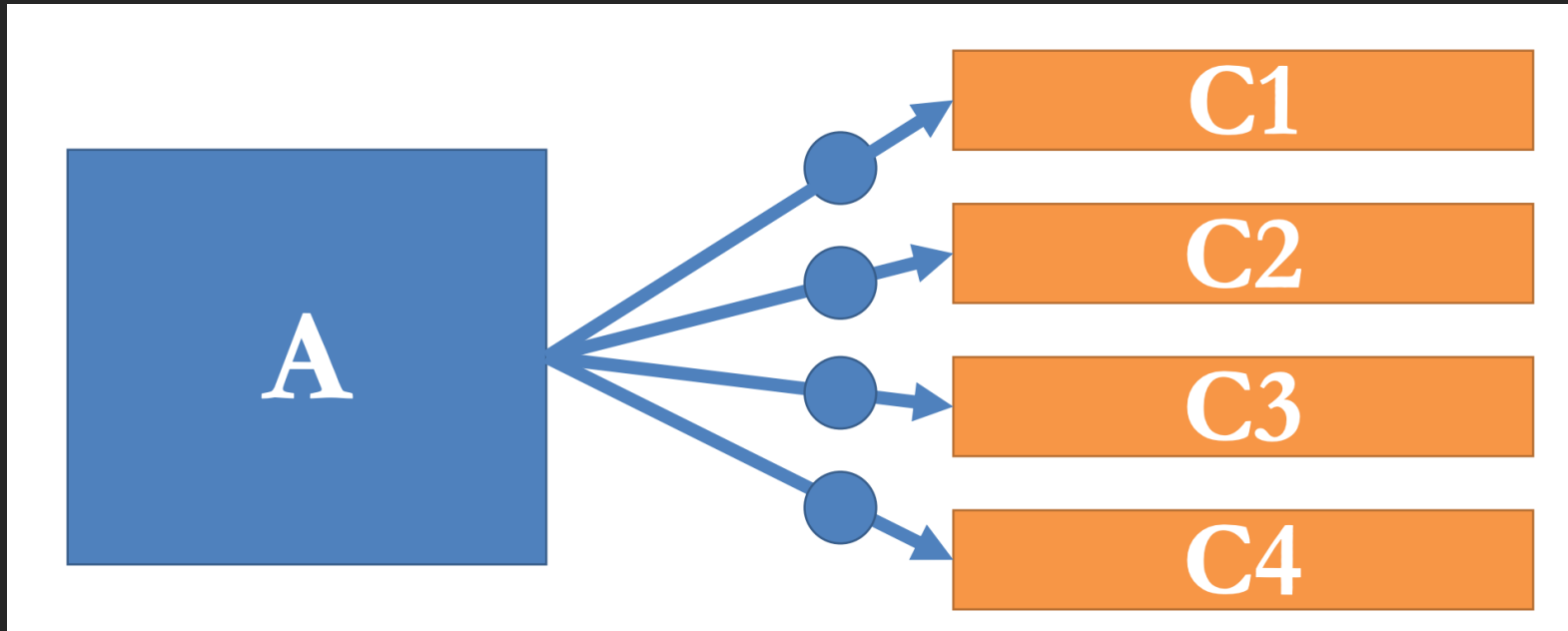
Machine B requests data from Machine A



Communication patterns

- Broadcast

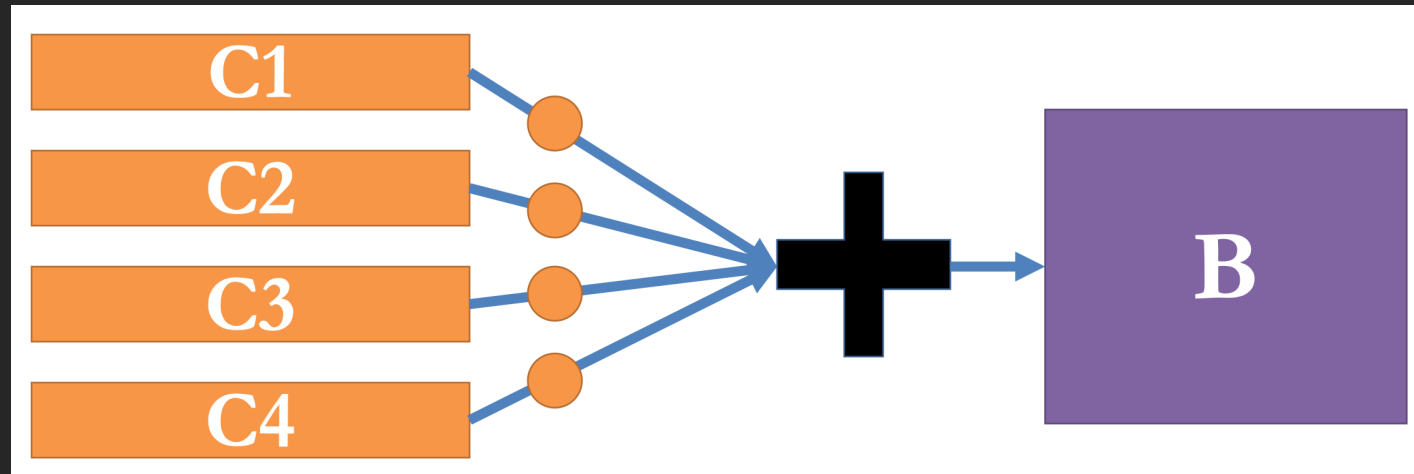
Machine A sends data to many machines



Communication patterns

- Reduce

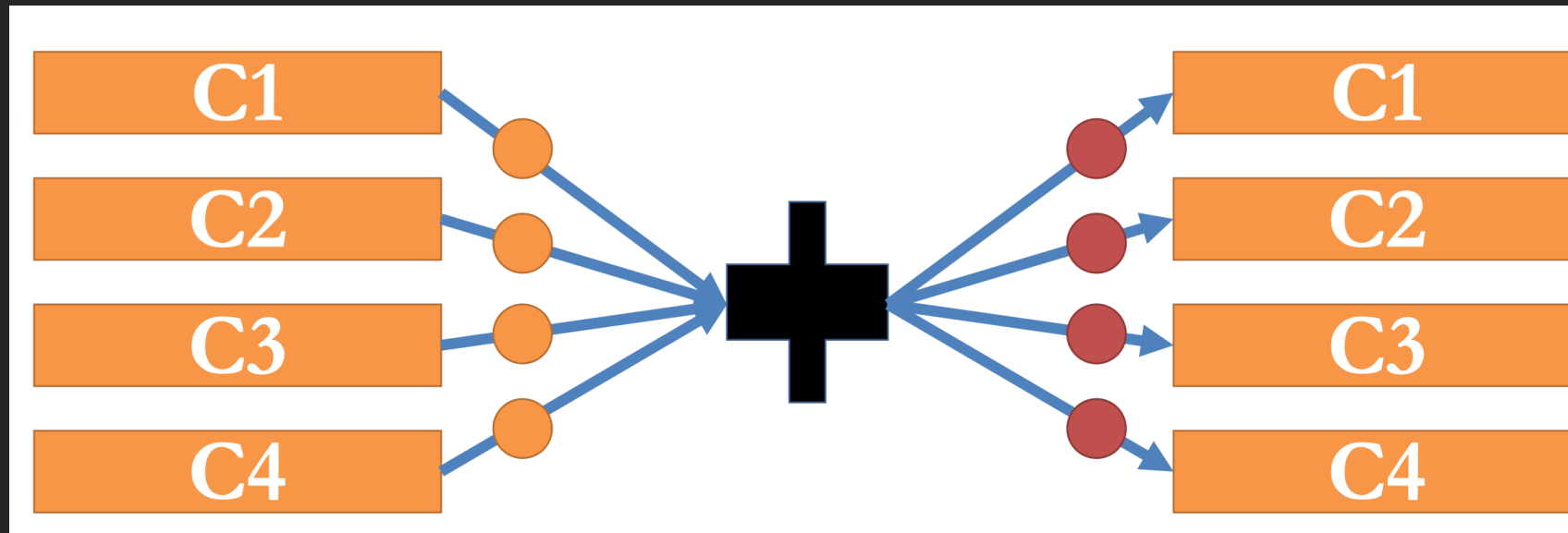
Compute some reduction (usually a sum) of data on multiple machines $C1, C2, \dots, Cn$ and materialize the result on one machine B



Communication patterns

- All-reduce

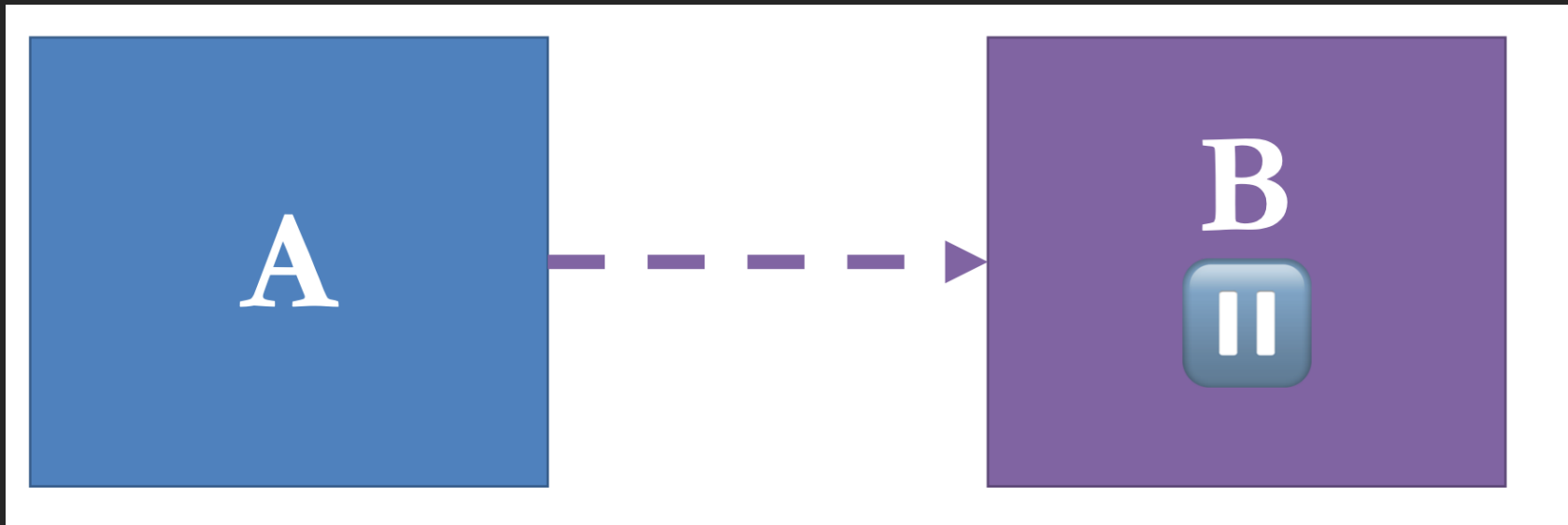
Compute some reduction (usually a sum) of data on multiple machines $C1, C2, \dots, Cn$ and materialize the result on all those machines



Communication patterns

- Wait

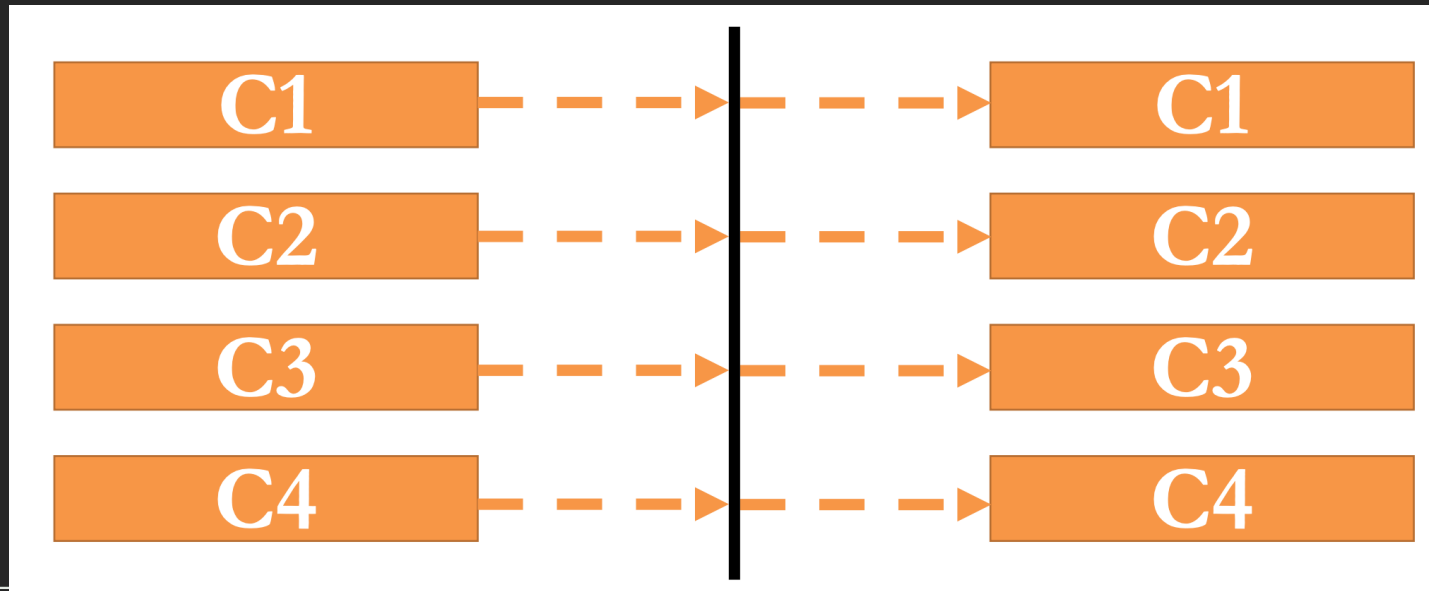
One machine pauses its computation and waits on a signal from another machine



Communication patterns

- Barrier

Many machines wait until all those machines reach a point in their execution, then continue from there



Overlapping computation and communication

- Communication over the network can be slow

we want to hide this latency

- An important principle of distributed computing is overlapping computation and communication

For the best performance, we want our workers to still be doing useful work while communication is going on

instead of stop and wait for the communication to finish

Stochastic Gradient Descent (SGD) with All-reduce

- All-reduce gives us a simple way of running learning algorithms such as SGD in a distributed fashion.
 - *Epoch means one pass over the full training set*
 - *Batch means that you use all your data to compute the gradient during one iteration.*
 - *Minibatch means you only take a subset of all your data during one iteration.*

$$w_{t+1} = w_t - \alpha_t \cdot \frac{1}{B} \sum_{b=1}^B \nabla f_{i_{b,t}}(w_t),$$

Stochastic Gradient Descent (SGD) with All-reduce

- If there are M worker machines such that $B = M \cdot B'$

$$w_{t+1} = w_t - \alpha_t \cdot \frac{1}{M} \sum_{m=1}^M \frac{1}{B'} \sum_{b=1}^{B'} \nabla f_{i_{m,b,t}}(w_t).$$

- Now, we assign the computation of the sum when $m = 1$ to worker 1, $m = 2$ to worker 2, etc.
 - After all the gradients are computed, we can perform the outer sum with an all-reduce operation
 - After this all-reduce, the whole sum (which is essentially the minibatch gradient) will be present on all the machines
so each machine can now update its copy of the parameters in the same way
-

Stochastic Gradient Descent (SGD) with All-reduce

Algorithm 1 Distributed SGD with All-Reduce

input: loss function examples $f_1, f_2, \dots$, number of machines M , per-machine minibatch size B'

input: learning rate schedule α_t , initial parameters w_0 , number of iterations T

for $m = 1$ **to** M **run in parallel on machine** m

load w_0 from algorithm inputs

for $t = 1$ **to** T **do**

select a minibatch $i_{m,1,t}, i_{m,2,t}, \dots, i_{m,B',t}$ of size B'

compute $g_{m,t} \leftarrow \frac{1}{B'} \sum_{b=1}^{B'} \nabla f_{i_{m,b,t}}(w_{t-1})$

all-reduce across all workers to compute $G_t = \sum_{m=1}^M g_{m,t}$

update model $w_t \leftarrow w_{t-1} - \frac{\alpha_t}{M} \cdot G_t$

end for

end parallel for

return w_T (from any machine)

Stochastic Gradient Descent (SGD) with All-reduce

- The algorithm is easy to reason about, since it's statistically equivalent to minibatch SGD.
 - We can use the same hyperparameters
 - The algorithm is easy to implement
all the worker machines have the same role and it runs on top of standard distributed computing primitives.
-

Stochastic Gradient Descent (SGD) with All-reduce

- Drawbacks

While the communication for the all-reduce is happening, the workers are idle

We're not overlapping computation and communication.

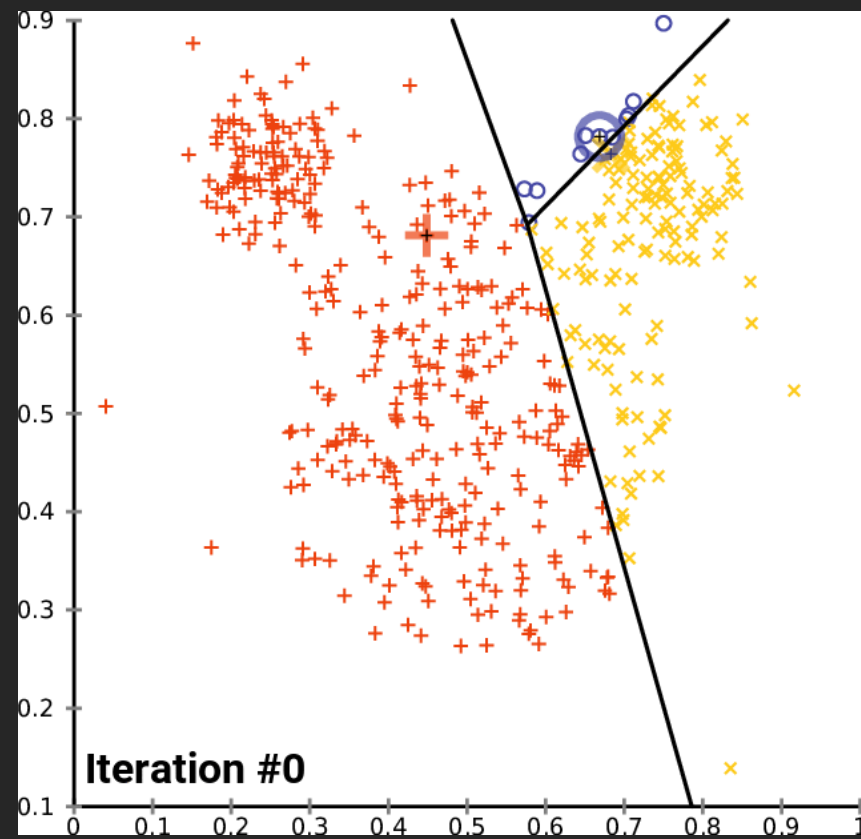
k-means algorithm

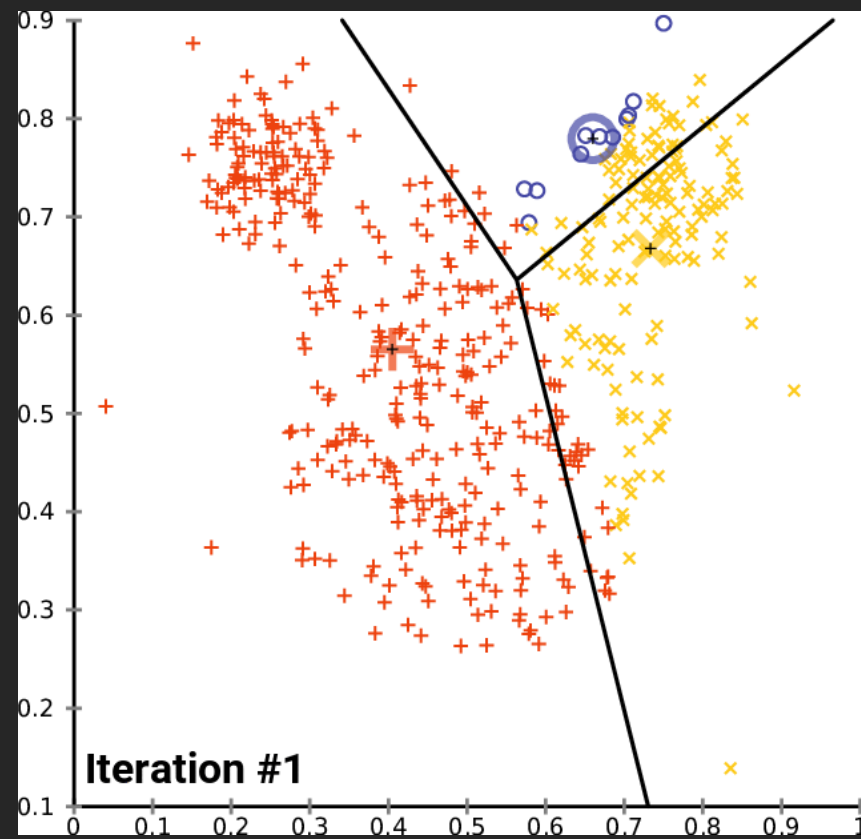
Algorithm 1.1 k-means clustering algorithm.

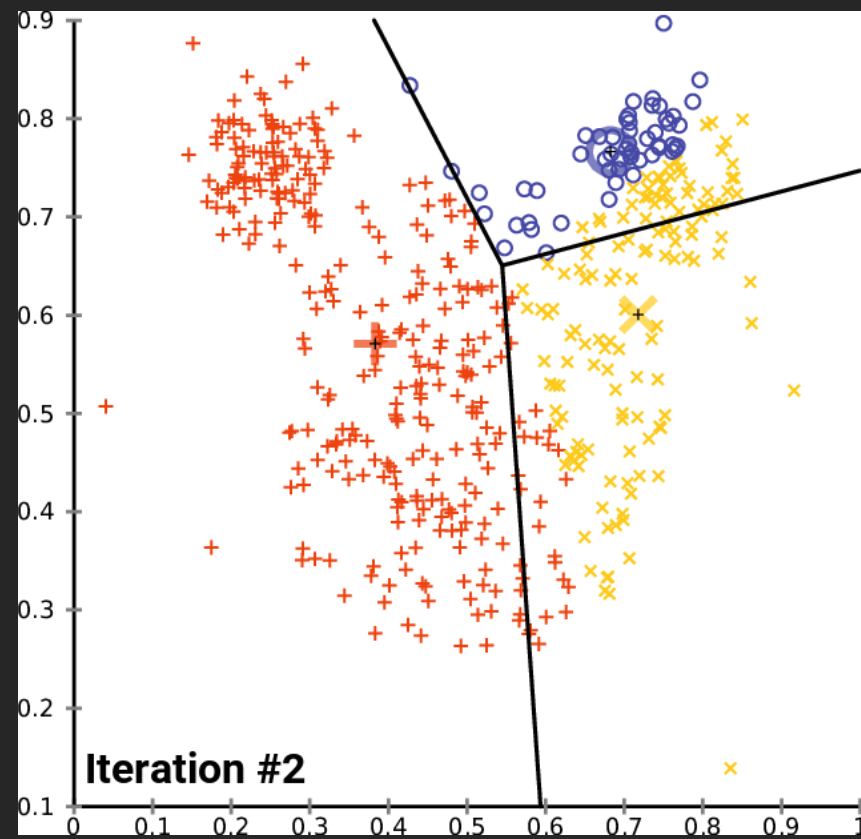
Input: Data $\mathcal{X} = \{\mathbf{x}_1, \dots, \mathbf{x}_n\}$, the order k , MAX number of allowed iterations

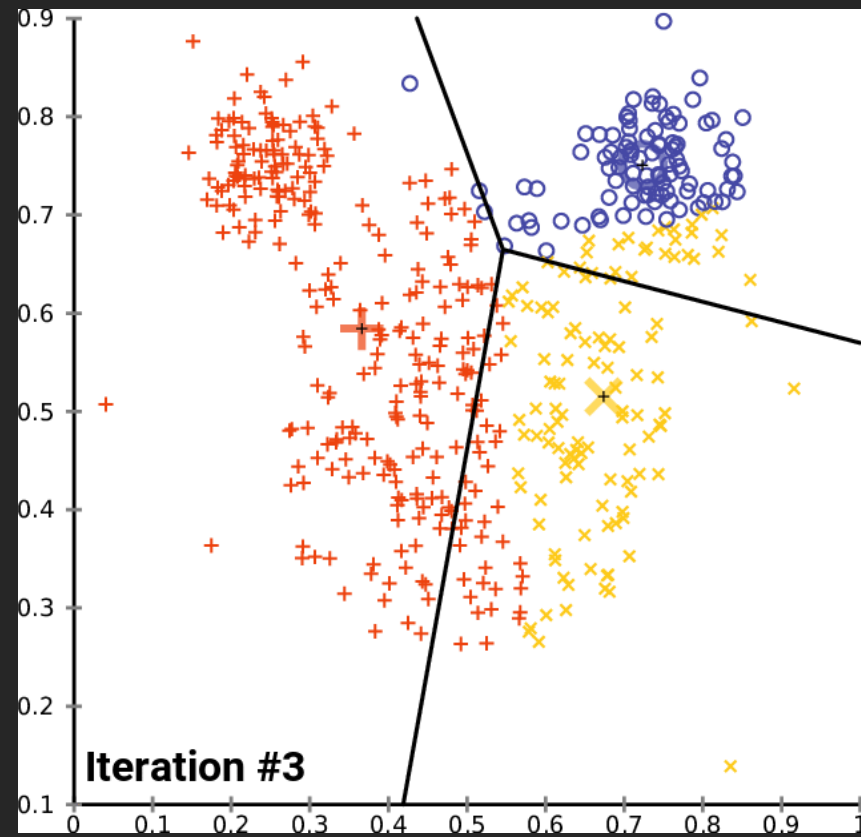
Output: A partition $\mathcal{P} = \{\mathcal{C}_1, \dots, \mathcal{C}_K\}$

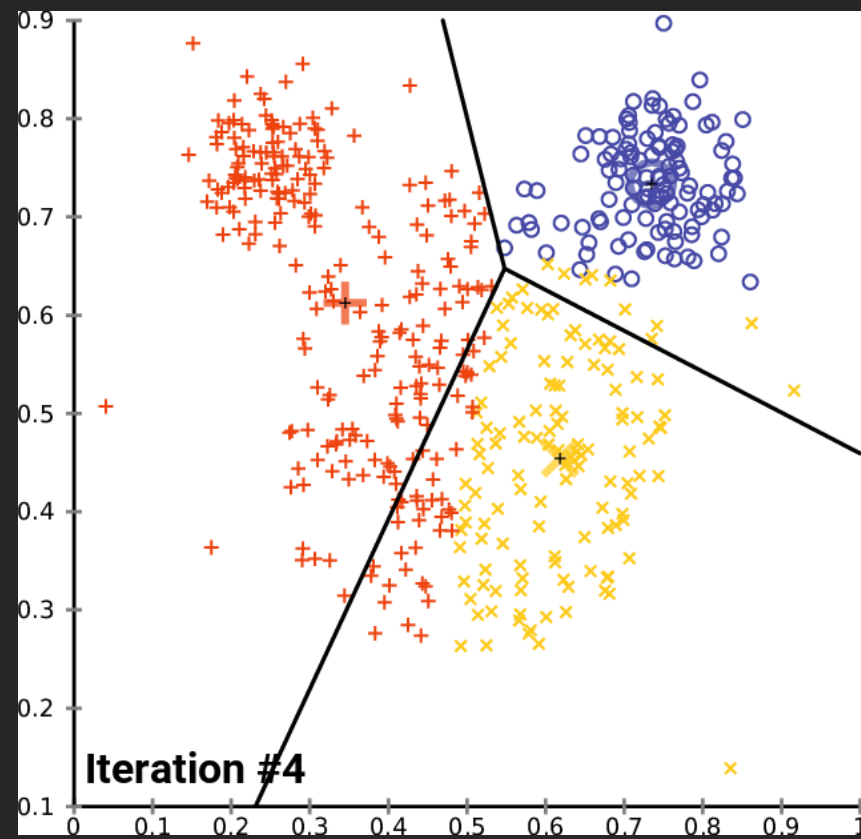
```
1:  $t = 0, \mathcal{P} = \emptyset$ 
2: Randomly initialize  $\mu_i, i = 1, \dots, K$ 
3: loop
4:    $t+ = 1$ 
5:   Assignment Step: assign each sample  $\mathbf{x}_j$  to the cluster with the nearest representative
6:    $\mathcal{C}_i^{(t)} = \{\mathbf{x}_j : d(\mathbf{x}_j, \mu_i) \leq d(\mathbf{x}_j, \mu_h) \text{ for all } h = 1, \dots, K\}$ 
7:   Update Step: update the representatives
8:    $\mu_i^{(t+1)} = \frac{1}{|\mathcal{C}_i^{(t)}|} \sum_{\mathbf{x}_j \in \mathcal{C}_i} \mathbf{x}_j$ 
9:   Update the partition with the modified clusters:
      $\mathcal{P}^t = \{\mathcal{C}_1^{(t)}, \dots, \mathcal{C}_K^{(t)}\}$ 
10:  if  $t \geq \text{MAX}$  OR  $\mathcal{P}^t = \mathcal{P}^{t-1}$  then
11:    return  $\mathcal{P}^t$ 
12:  end if
13: end loop
```

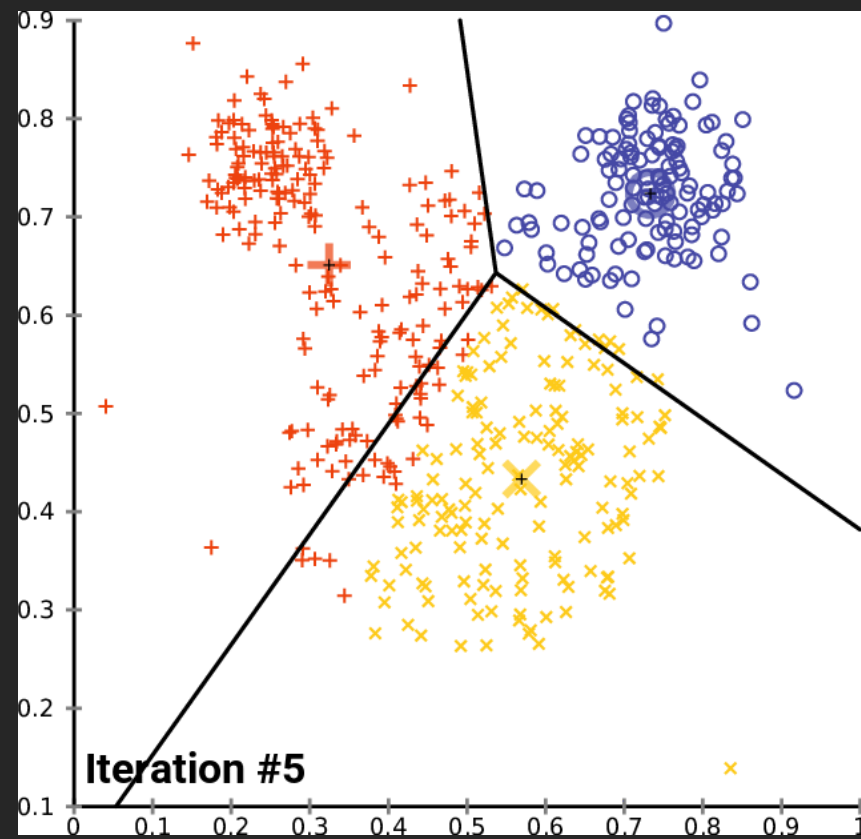


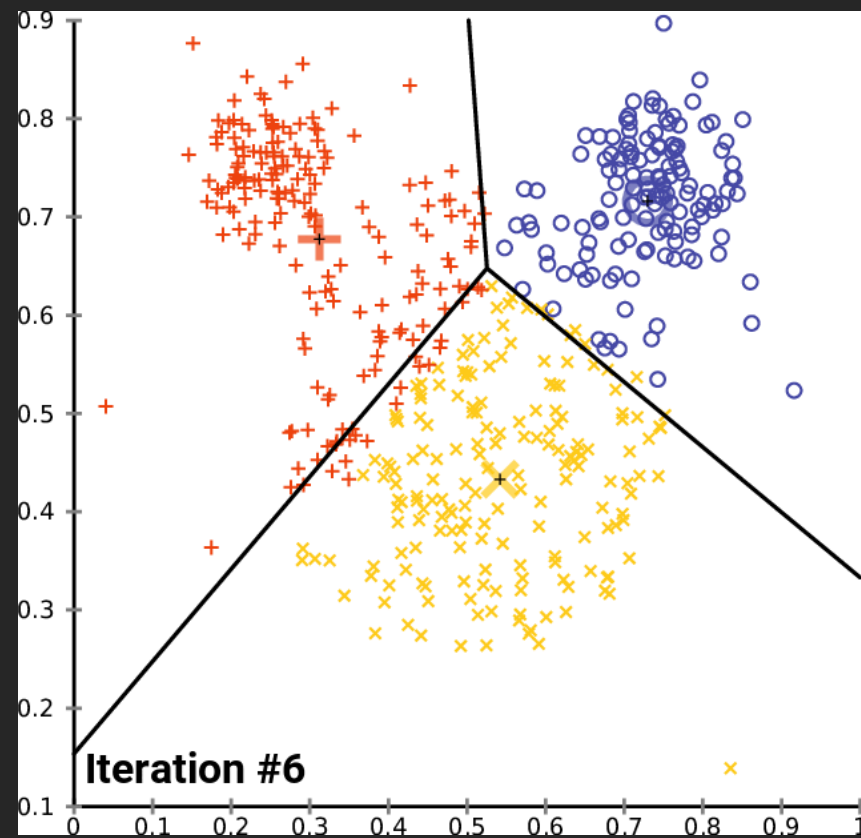


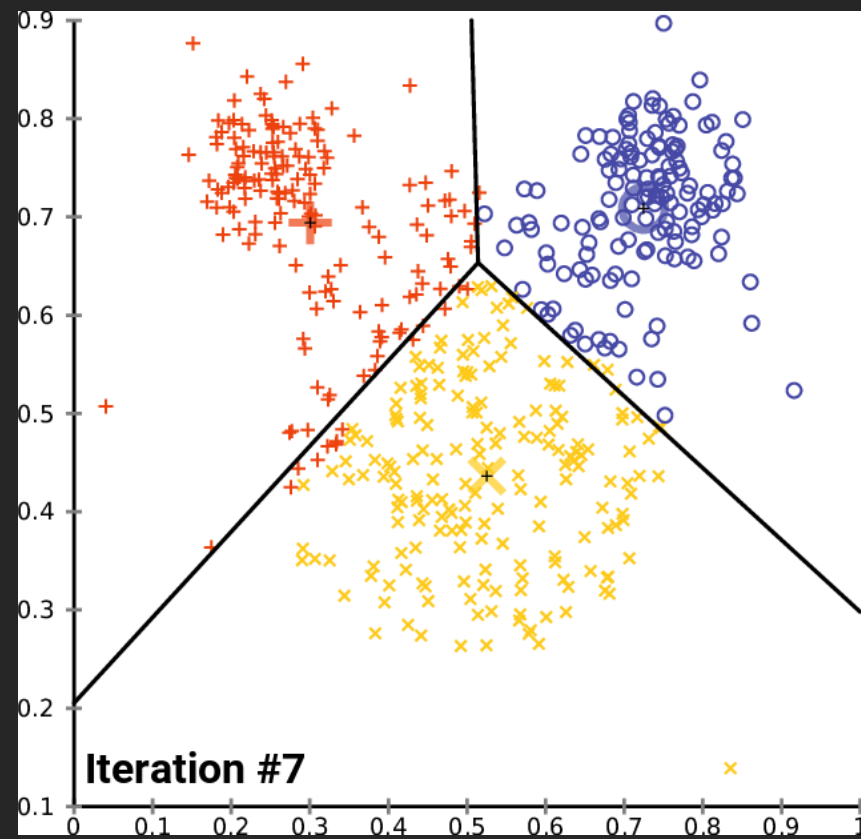


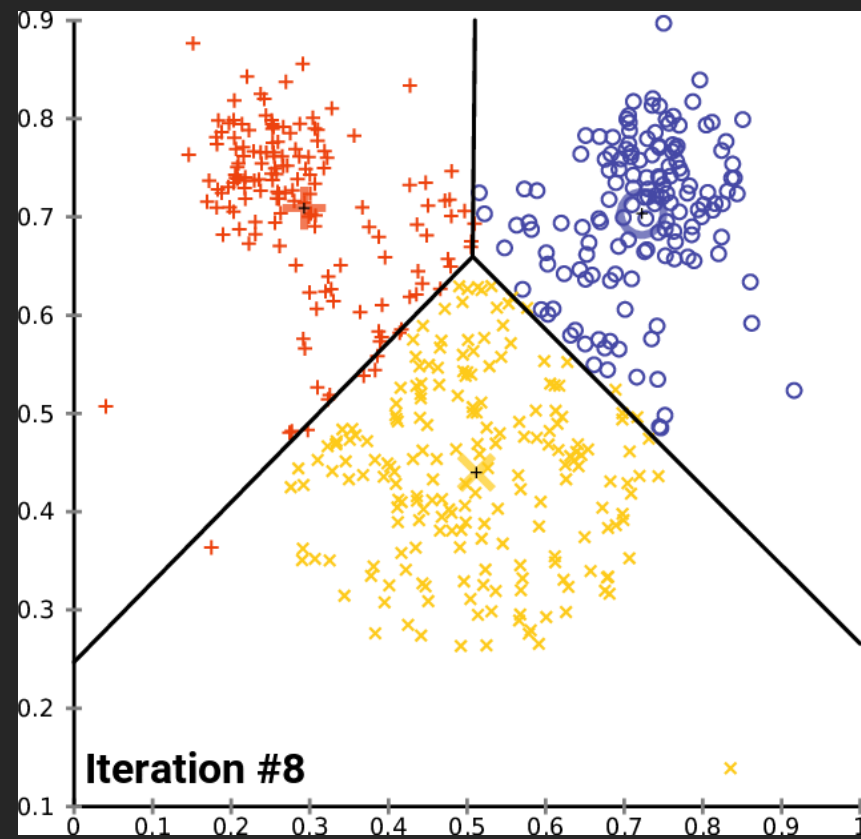


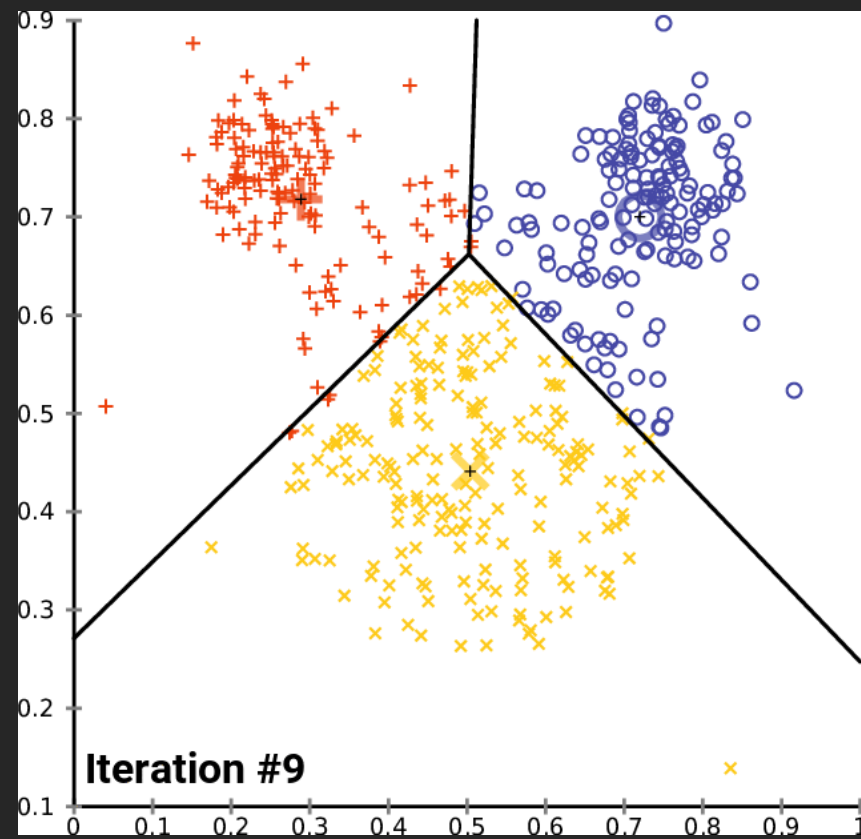


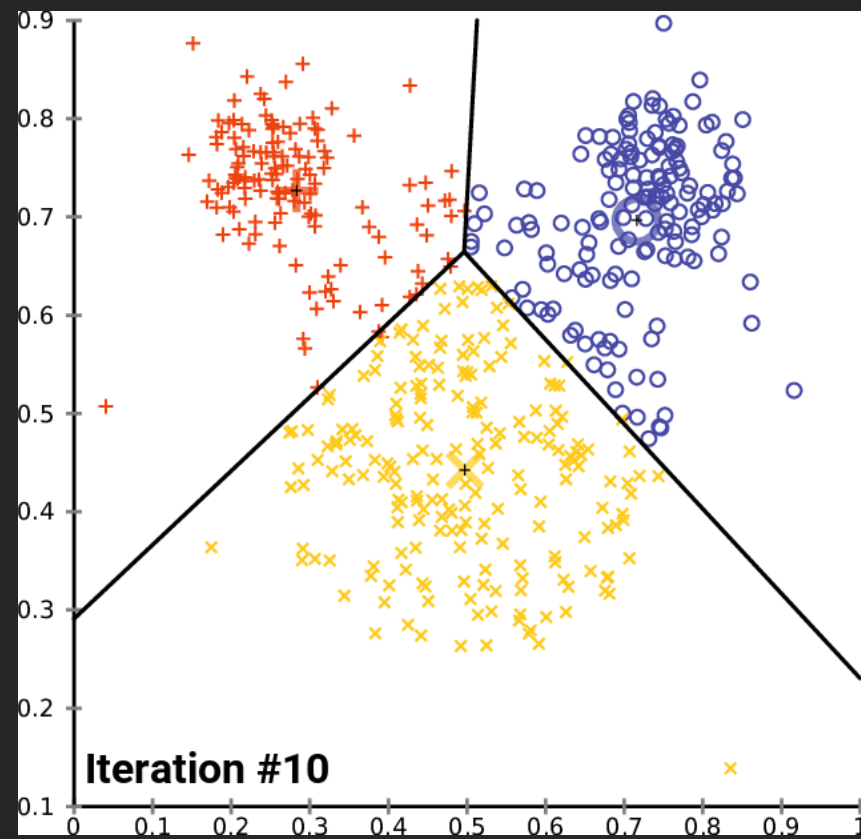


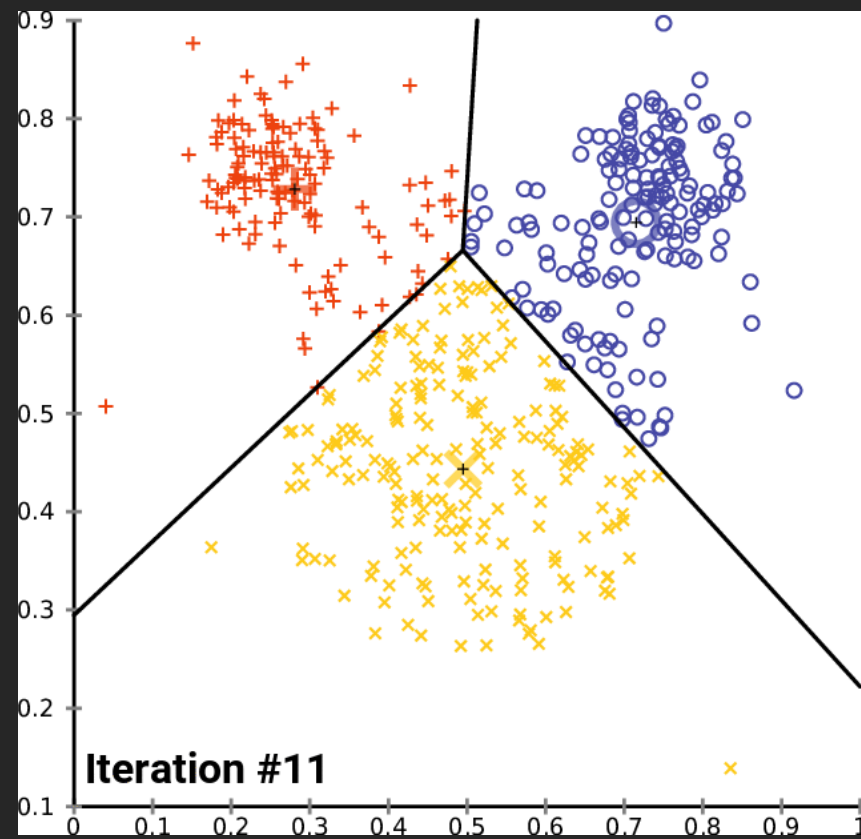


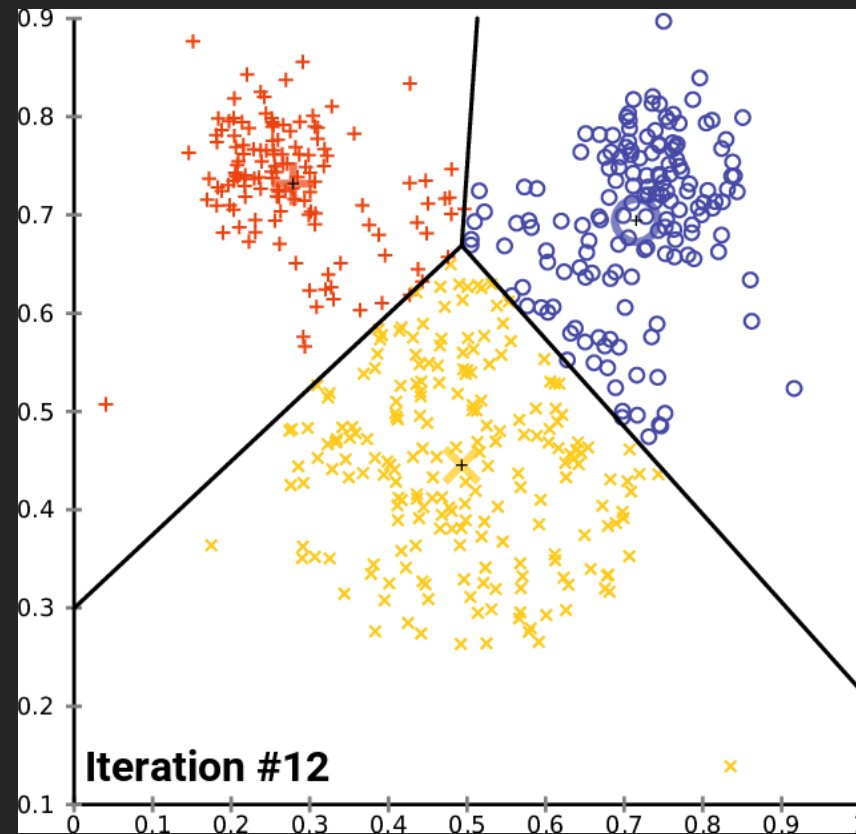


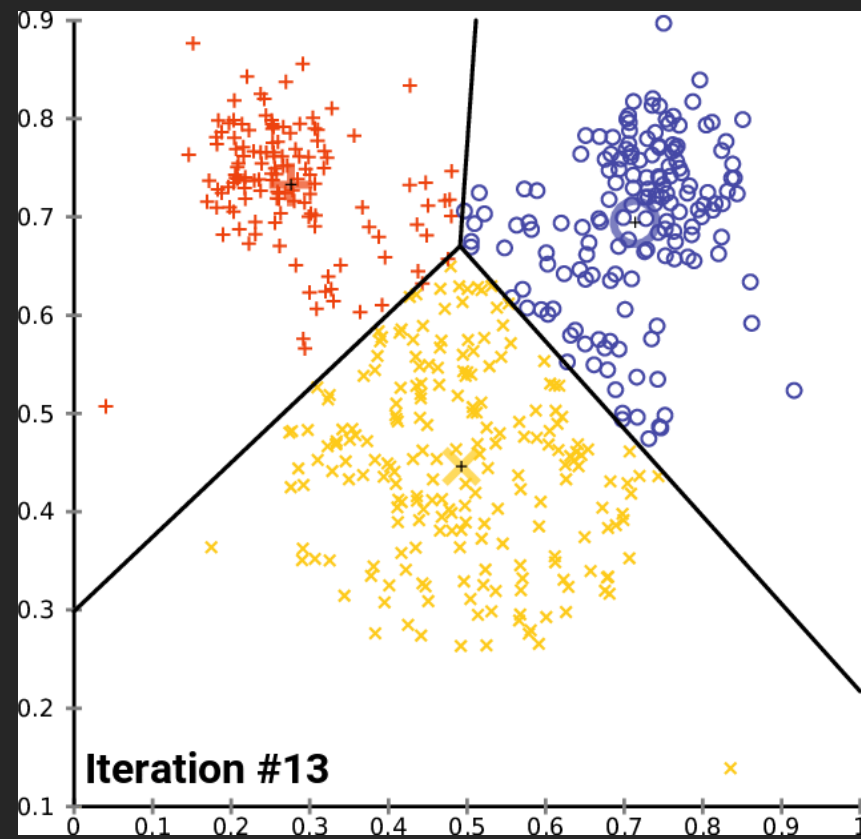


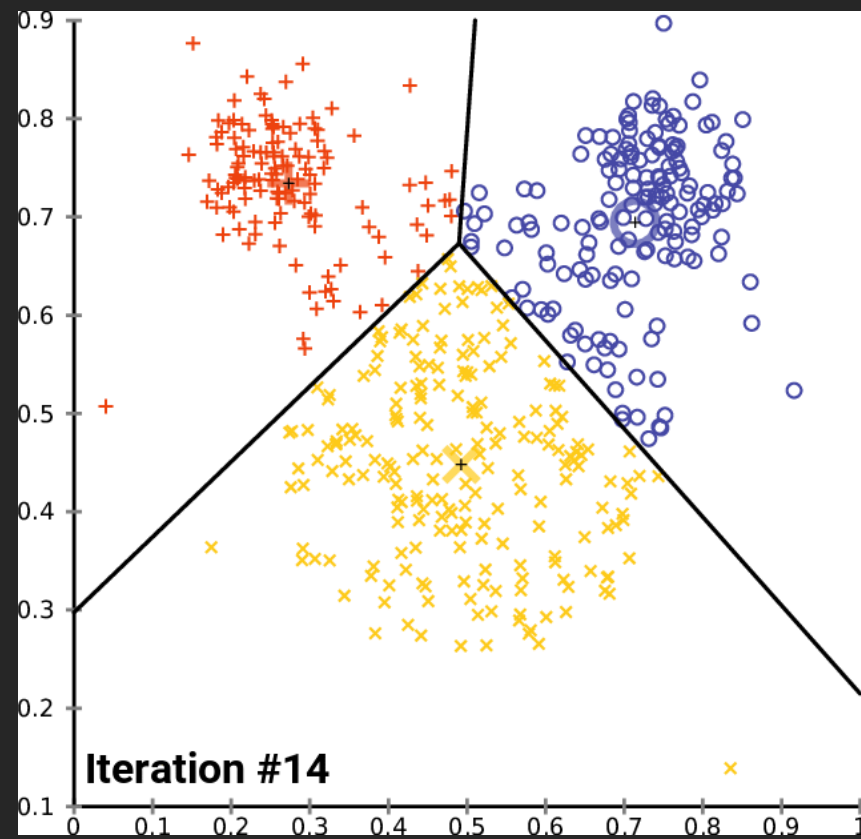












Parallel K-means

- MapReduce implementation

Split into three phases

- Map

Compute distances between points and the K centroids

Assign points to the clusters

Send intermediate data to Combiner (membership)

- Combine

Computes the centroid of each cluster

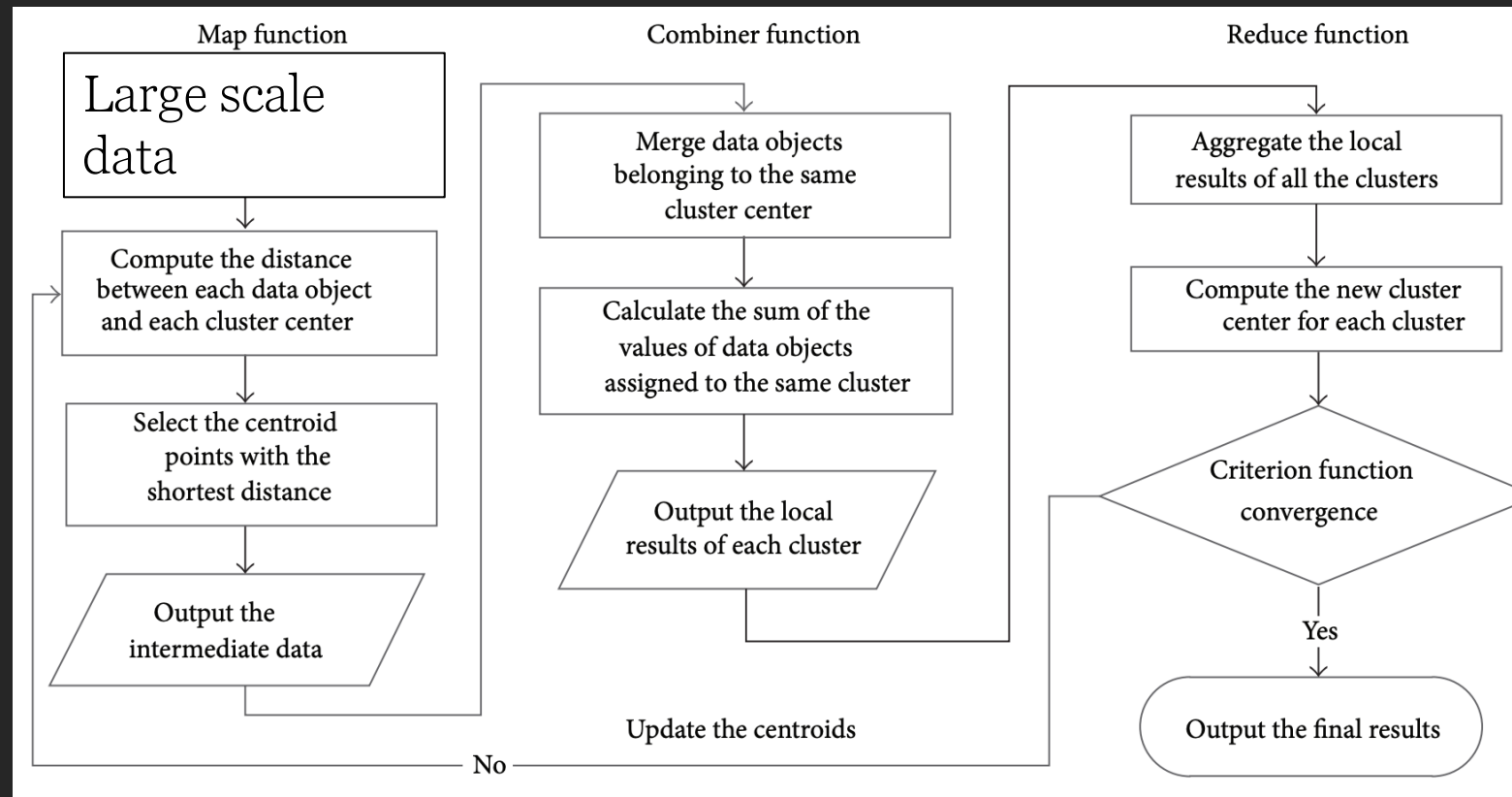
Sends the local sums of the values across dimensions and the number of samples

- Reduce

Test convergence

Update the cluster centroids

Parallel K-means



Parallel K-means

Input:

key: the offset,
value: the sample,
centroids: the global variable.

Output: $\langle key_1, value_1 \rangle$,

key_1 : the index of the closest centroid,
 $value_1$: the information of sample.

- (1) Construct a global variable *centroids* including the information of the closet centroid point;
- (2) Construct the sample *examples* to extract the data objects from *value*;
- (3) *min_Dis* = *Double.MAX_VALUE*;
- (4) *index* = -1;
- (5) **for** *i* = 0 to *centroids.length* **do**
- (6) *distance* = *DistanceFunction(examples, centroids[i])*;
- (7) **if** *distance* < *min_Dis* **then**
- (8) *min_Dis* = *distance*;
- (9) *index* = *i*;
- (10) **end if**
- (11) **end for**
- (12) *index* = key_1 ;
- (13) Construct $value_1$ as a string consisting of the values of different dimensions;
- (14) **return** $\langle key_1, value_1 \rangle$ pairs;

ALGORITHM 1: Map(*key*, *value*).

Parallel K-means

Input:

key_1 : the index of the cluster,
 $medi$: the list of the samples assigned to the same cluster.

Output: $\langle key_2, value_2 \rangle$,

key_2 : the index of the cluster,
 $value_2$: the sum of the values of the samples belonging to the same cluster and the number of samples.

- (1) Construct a counter num_s to record the number of samples in the same cluster;
- (2) Construct an array sum_v to record the sum of the values of different dimensions of the samples belonging to the same cluster (i.e., the samples in the list $medi$);
- (3) Construct the sample $examples$ to extract the data objects from $medi.next()$, and the $dimensions$ to obtain the dimension of the original data object;
- (4) $num_s = 0$;
- (5) **while** ($medi.hasNext()$) **do**
- (6) $CurrentPoint = medi.next()$;
- (7) num_s++ ;
- (8) **for** $i = 0$ to $dimensions$ **do**
- (9) $sum_v[i] += CurrentPoint.point[i]$;
- (10) //Calculate the sum of the values of each dimension of $examples$
- (11) **end for**
- (12) **for** $i = 0$ to $dimensions$ **do**
- (13) $mean[i] = sum_v[i] / num_s$;
- (14) //Compute the mean value of the samples for each cluster
- (15) **end for**
- (16) **end while**
- (17) $index = key_2$;
- (18) Construct $value_2$ as a string containing the sum of the values of each dimension $sum_v[i]$ and the number of samples num_s ;
- (19) **return** $\langle key_2, value_2 \rangle$ pairs;

ALGORITHM 2: Combiner($key_1, medi$).

Parallel K-means

Input:

key_2 : the index of the cluster,
 $medi$: the list of the local sums from different clusters.

Output: $\langle key_3, value_3 \rangle$,

key_3 : the index of the cluster,
 $value_3$: the new cluster center.

- (1) Construct a counter Num to record the total number of samples belonging to the same cluster;
- (2) Construct an array sum_v to record the sum of the values of different dimensions of the samples in the same cluster (i.e., the samples in the list $medi$);
- (3) Construct the sample *examples* to extract the data objects from $medi.next()$, and the *dimensions* to obtain the *dimension* of the original data object;
- (4) $Num = 0$;
- (5) **while** ($medi.hasNext()$) **do**
- (6) $CurrentPoint = medi.next()$;
- (7) $Num += num_s$;
- (8) **for** $i = 0$ to $dimensions$ **do**
- (9) $sum_v[i] += CurrentPoint.point[i]$;
- (10) **end for**
- (11) **for** $i = 0$ to $dimensions$ **do**
- (12) $mean[i] = sum_v[i] / Num$;
- (13) //Obtain the new cluster center
- (14) **end for**
- (15) **end while**
- (16) $index = key_3$;
- (17) Construct $value_3$ as a string composed of the new cluster center;
- (18) **return** $\langle key_3, value_3 \rangle$ pairs;

ALGORITHM 3: Reduce($key_2, medi$).